

Three-Schema Architecture

- Defines DBMS schemas at **three** levels:
 - **Internal schema** at the internal level to describe physical storage structures and access paths (e.g indexes).
 - Typically uses a **physical** data model.
 - **Conceptual schema** at the conceptual level to describe the structure and constraints for the whole database for a community of users.
 - Uses a **conceptual** or an **implementation** data model.
 - **External schemas** at the external level to describe the various user views.
 - Usually uses the same data model as the conceptual schema.

Physical Level:

- Describes **HOW data are actually stored**
- It deals with the data structures used in physical
- It is otherwise called as Internal level

Logical Level:

- **Describes WHAT data is stored in database** and their relationships among data
- This level is usually designed by using ER diagrams
- It is otherwise called as Conceptual Level, Schema level

External Level:

- **Only part of the database can be viewed** at a time but not the entire database
- Database has many views
- It is otherwise called as Subschema level, View level

Levels of Abstraction

- **Physical level:** describes how a record (e.g., customer) is stored.
- **Logical level:** describes data stored in database, and the relationships among the data.

type *customer* = **record**

```
customer_id : string;  
customer_name : string;  
customer_street : string;  
customer_city : string;
```

end;

- **View level:** application programs hide details of data types. Views can also hide information (such as an employee's salary) for security purposes.

Data Independence

- Changing data or structure in one level without effecting higher levels
- Applications insulated from how data is structured and stored.
- Logical data independence: Protection from changes in *logical* structure of data.
- Physical data independence: Protection from changes in *physical* structure of data.

✉ *One of the most important benefits of using a DBMS!*

Physical Data Independence

External Level

Logical Level

Physical Level



Logical Data Independence

External Level

Logical Level



ID	Song Title	Category	Solo Type
1	Pachelbel's Canon	Classical	Instrumental
2	Handel's Hallelujah	Choral	Vocal
3	Enya - Only Time	New Age	Vocal

Physical Level

Data Models

- A **collection of tools** for describing
 - Data
 - Data relationships
 - Data semantics
 - Data constraints
- **Relational model**
- **Entity-Relationship** data model (mainly for database design)
- **Object-based** data models (Object-oriented and Object-relational)
- **Semi structured** data model (XML)
- Other older models:
 - **Network model**
 - **Hierarchical model**

- Both **network** and **hierarchical models** are involved with more number of pointers.
- A single misplaced pointer corrupts the entire database.
- Relational Model is free from pointers

Relational Model

Table name

Product

Attribute names

PName	Price	Category	Manufacturer
Gizmo	\$19.99	Gadgets	GizmoWorks
Powergizmo	\$29.99	Gadgets	GizmoWorks
SingleTouch	\$149.99	Photography	Canon
MultiTouch	\$203.99	Household	Hitachi

Tuples or rows

- A tuple/ row is a combination of different column values of one real world object.
- Number of columns represents ***degree*** of the table
- Number of tuples represents ***cardinality*** of the table
- Tuple is identified uniquely by using primary key

Note: The term “Relation” is abstract and “Table” is concrete

Instances and Schemas

- Similar to types and variables in programming languages
- **Schema** – the logical structure of the database
 - Example: The database consists of information about a set of customers and accounts and the relationship between them)
 - Analogous to type information of a variable in a program
 - **Physical schema:** database design at the physical level
 - **Logical schema:** database design at the logical level
- **Instance** – the actual content of the database at a particular point in time

Example of a Database Schema

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Figure 2.1

Schema diagram for the database in Figure 1.2.

Example of a Database Schema

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2

A database that stores student and course information.

Example: University Database

- **Conceptual schema:**

- *Students(sid: string, name: string, login: string, age: integer, gpa:real)*
- *Courses(cid: string, cname:string, credits:integer)*
- *Enrolled(sid:string, cid:string, grade:string)*

- **Physical schema:**

- Relations stored as unordered files.
- Index on first column of Students.

- **External Schema (View):**

- *Course_info(cid:string,enrollment:integer)*

DATA BASE LANGUAGE

Data Definition Language (DDL)

- Specification notation for **defining the database schema**

Example: **create table** *account* (
 account_number **char**(10),
 branch_name **char**(10),
 balance **integer**)

- DDL compiler **generates a set of tables** stored in a **data dictionary**
- **Data dictionary** contains **metadata** (i.e., data about data)
 - Database schema
 - Data *storage and definition* language
 - Specifies the storage structure and access methods used
 - Integrity constraints
 - Domain constraints
 - Referential integrity (e.g. *branch_name* must correspond to a valid branch in the *branch* table)
 - Authorization

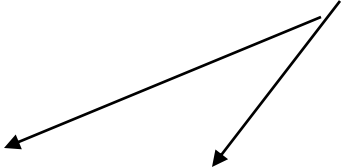
Data Manipulation Language (DML)

- Language for **accessing** and **manipulating** the **data organized** by the appropriate **data model**
 - **DML** also known as **query language**
- Two classes of languages
 - **Procedural** – user specifies what data is required and how to get those data
 - **Declarative (nonprocedural)** – user specifies what data is required without specifying how to get those data
- SQL is the most widely used query language

Relational Model

- Example of tabular data in the relational model

Attributes



<i>customer_id</i>	<i>customer_name</i>	<i>customer_street</i>	<i>customer_city</i>	<i>account_number</i>
192-83-7465	Johnson	12 Alma St.	Palo Alto	A-101
192-83-7465	Johnson	12 Alma St.	Palo Alto	A-201
677-89-9011	Hayes	3 Main St.	Harrison	A-102
182-73-6091	Turner	123 Putnam St.	Stamford	A-305
321-12-3123	Jones	100 Main St.	Harrison	A-217
336-66-9999	Lindsay	175 Park Ave.	Pittsfield	A-222
019-28-3746	Smith	72 North St.	Rye	A-201

A Sample Relational Database

<i>customer_id</i>	<i>customer_name</i>	<i>customer_street</i>	<i>customer_city</i>
192-83-7465	Johnson	12 Alma St.	Palo Alto
677-89-9011	Hayes	3 Main St.	Harrison
182-73-6091	Turner	123 Putnam Ave.	Stamford
321-12-3123	Jones	100 Main St.	Harrison
336-66-9999	Lindsay	175 Park Ave.	Pittsfield
019-28-3746	Smith	72 North St.	Rye

(a) The *customer* table

<i>account_number</i>	<i>balance</i>
A-101	500
A-215	700
A-102	400
A-305	350
A-201	900
A-217	750
A-222	700

(b) The *account* table

<i>customer_id</i>	<i>account_number</i>
192-83-7465	A-101
192-83-7465	A-201
019-28-3746	A-215
677-89-9011	A-102
182-73-6091	A-305
321-12-3123	A-217
336-66-9999	A-222
019-28-3746	A-201

(c) The *depositor* table

SQL

- **SQL**: widely used non-procedural language
 - Example: Find the name of the customer with customer-id 192-83-7465

```
select    customer.customer_name
from      customer
where     customer.customer_id = '192-83-7465'
```
 - Example: Find the balances of all accounts held by the customer with customer-id 192-83-7465

```
select    account.balance
from      depositor, account
where     depositor.customer_id = '192-83-7465' and
           depositor.account_number =
           account.account_number
```

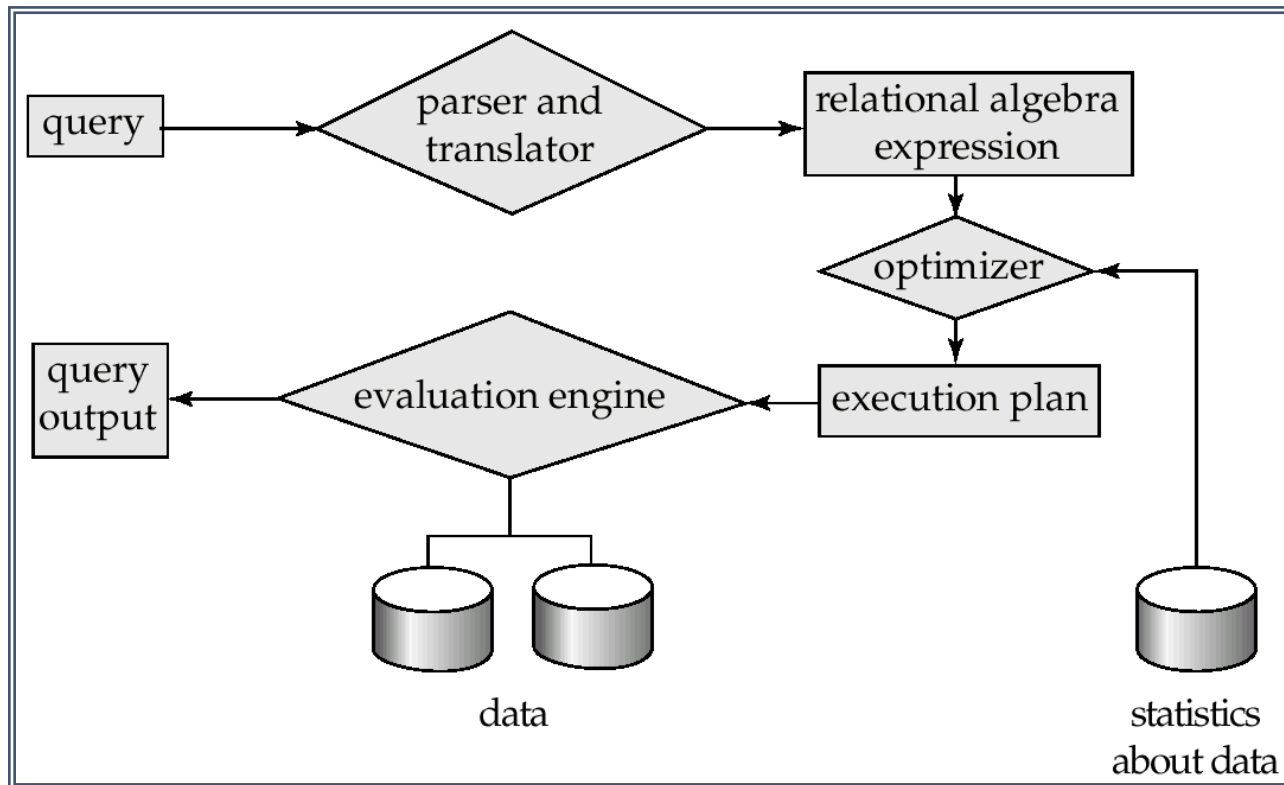
Tools Used in DBMS Architecture

Data dictionary / repository:

- Used to store schema descriptions and other information such as design decisions, application program descriptions, user information, usage standards, etc.
- **Active data dictionary** is accessed by DBMS software and users/DBA.
- **Passive data dictionary** is accessed by users/DBA only.

Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation



Database Architecture

The architecture of a database systems is greatly influenced by the underlying computer system on which the database is running:

- Centralized
- Client-server
- Parallel (multiple processors and disks)
- Distributed

Centralized and Client-Server DBMS Architectures

Centralized DBMS

- Combines everything into single system including- DBMS software, hardware, application programs, and user interface processing software.
- User can still connect through a remote terminal – however, all processing is done at centralized site.

A Physical Centralized Architecture

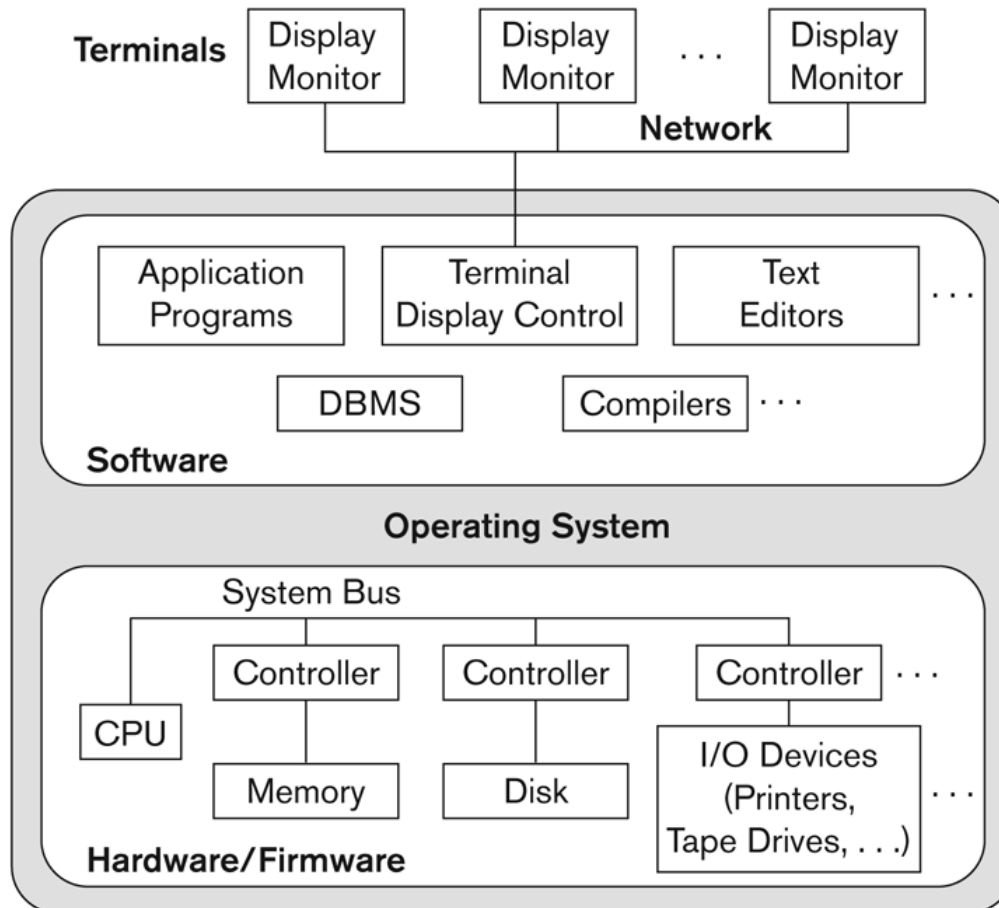


Figure 2.4
A physical centralized architecture.

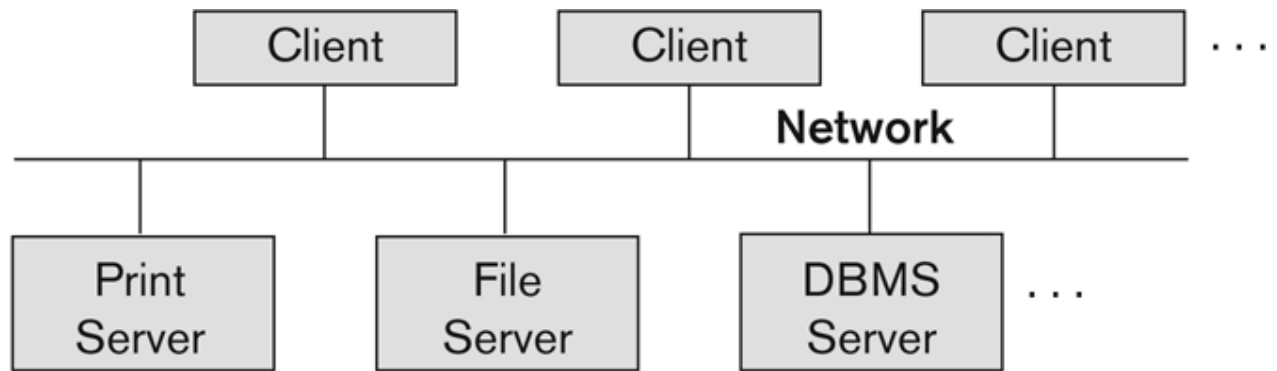
Basic 2-tier Client-Server Architectures

- Specialized Servers with Specialized functions
 - Print server
 - File server
 - DBMS server
 - Web server
 - Email server
- Clients can access the specialized servers as needed
- Ex. Games and Music Players
- Fast to access

Logical two-tier client server architecture

Figure 2.5

Logical two-tier
client/server
architecture.



Clients

- Provide appropriate interfaces through a client software module to access and utilize the various server resources.
- Clients may be diskless machines or PCs or Workstations with disks with only the client software installed.
- Connected to the servers via some form of a network.
 - (LAN: local area network, wireless network, etc.)

DBMS Server

- Provides database query and transaction services to the clients
- Relational DBMS servers are often called SQL servers, query servers, or transaction servers
- Applications running on clients utilize an Application Program Interface (**API**) to access server databases via standard interface such as:
 - ODBC: Open Database Connectivity standard
 - JDBC: for Java programming access
- Client and server must install appropriate client module and server module software for ODBC or JDBC

Two Tier Client-Server Architecture

- A client program may connect to several DBMSs, sometimes called the data sources.
- In general, data sources can be files or other non-DBMS software that manages data.
- Other variations of clients are possible: e.g., in some object DBMSs, more functionality is transferred to clients including data dictionary functions, optimization and recovery across multiple servers, etc.

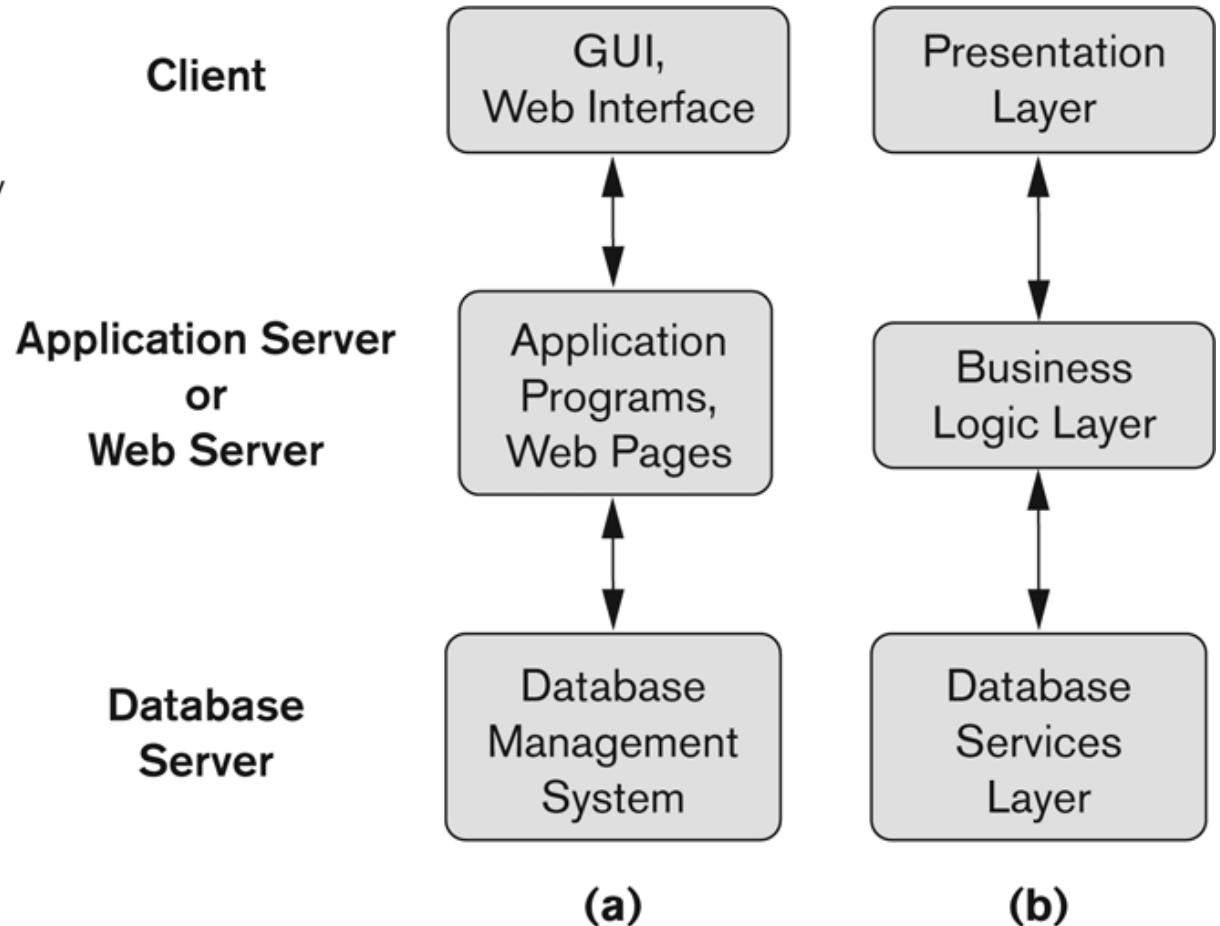
Three Tier Client-Server Architecture

- Common for Web applications
- Intermediate Layer called Application Server or Web Server:
 - Stores the web connectivity software and the business logic part of the application used to access the corresponding data from the database server
 - Acts like a conduit for sending partially processed data between the database server and the client.
- Three-tier Architecture Can Enhance Security:
 - Database server only accessible via middle tier
 - Clients cannot directly access database server

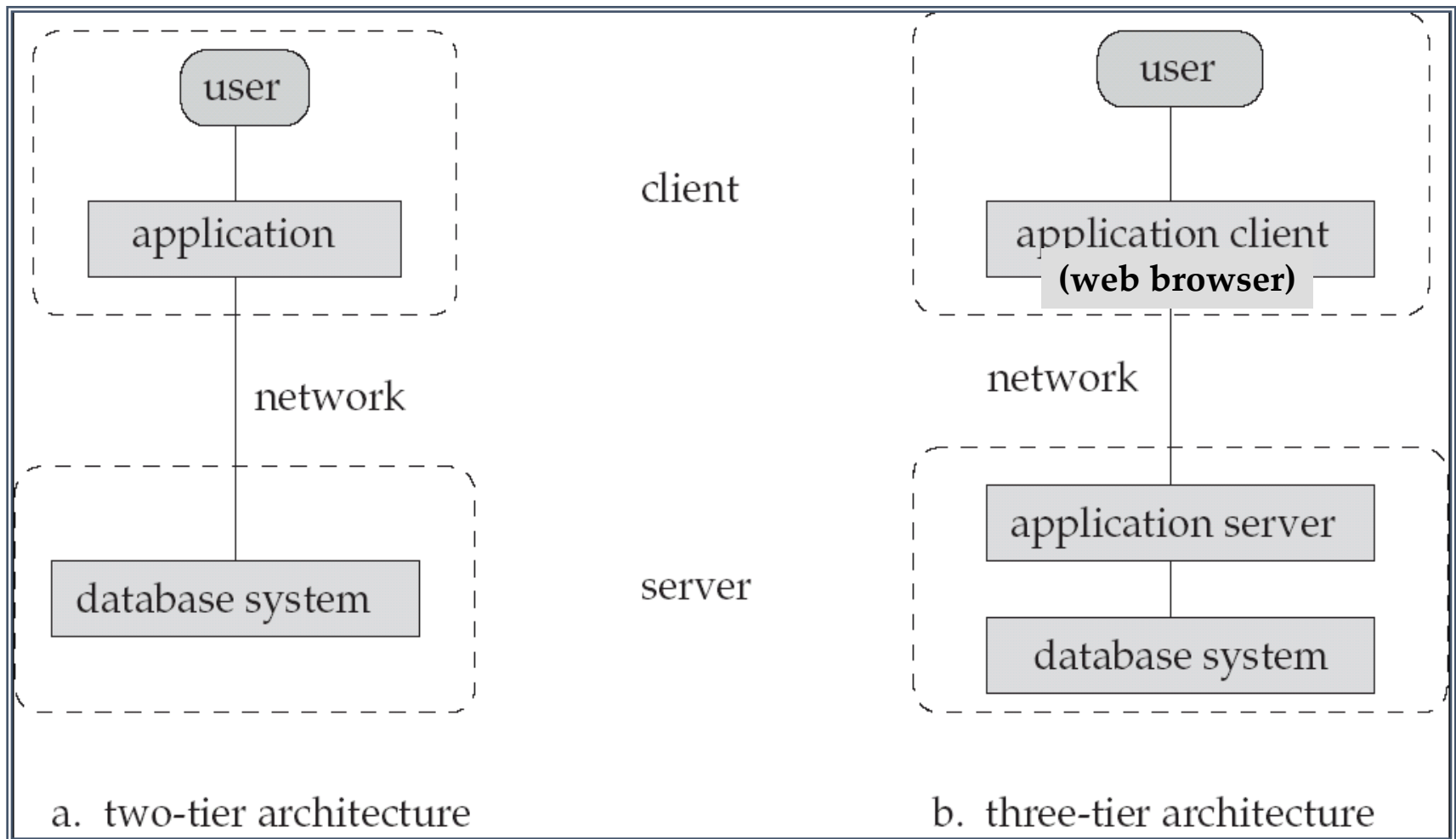
Three-tier client-server architecture

Figure 2.7

Logical three-tier client/server architecture, with a couple of commonly used nomenclatures.



Database Application Architectures



Old

Modern

Classification of DBMSs

- Based on the data model used
 - Traditional: Relational, Network, Hierarchical.
 - Emerging: Object-oriented, Object-relational.
- Other classifications
 - Single-user (typically used with personal computers) vs. multi-user (most DBMSs).
 - Centralized (uses a single computer with one database) vs. distributed (uses multiple computers, multiple databases)

Variations of Distributed DBMSs (DDBMSs)

- Homogeneous DDBMS
- Heterogeneous DDBMS
- Federated or Multidatabase Systems
- Distributed Database Systems have now come to be known as client-server based database systems because:
 - They do not support a totally distributed environment, but rather a set of database servers supporting a set of clients.

Cost considerations for DBMSs

- Cost Range: from free open-source systems to configurations costing millions of dollars
- Examples of free relational DBMSs: MySQL, PostgreSQL, others
- Commercial DBMS offer additional specialized modules, e.g. time-series module, spatial data module, document module, XML module
 - These offer additional specialized functionality when purchased separately
 - Sometimes called cartridges (e.g., in Oracle) or blades
- Different licensing options: site license, maximum number of concurrent users (seat license), single user, etc.

Classification of Database Systems

centralized database system: the DBMS and database are stored at a single site that is used by several other systems too

distributed database system: the actual database and the DBMS software are distributed from various sites that are connected by a computer network

heterogeneous distributed database system: different sites might use different DBMS software, but there is additional common software to support data exchange between these sites

homogeneous distributed database systems: use the same DBMS software at multiple sites

Classification of Database Systems

multiuser database system: a database management system which supports multiple users concurrently

object-oriented data model: a database management system in which information is represented in the form of objects as used in object-oriented programming

single-user database system: a database management system which supports one user at a time

traditional models: data models that preceded the relational model